Computación y estructuras discretas I

Unidad 3: Estructuras Discretas no recursivas y análisis de algoritmos – Generics

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas
Universidad ICESI
Material adaptado del material original del profesor Andrés Aristizábal

2026

Tabla de contenido

1. Generics

1.1 Introducción
1.2 Tipos genéricos
1.3 Tipos raw
1.4 Métodos genéricos
1.5 Wildcards
1.6 Lo no permitido por Generics
1.7 Ejemplos
Ejemplo: Pila Generica
Ejemplo: Pila con Caja Generica
Ejemplo: Pila con Caja Generica y Wildcard

Contenido

1. Generics

1.1 Introducción
1.2 Tipos genéricos
1.3 Tipos raw
1.4 Métodos genéricos
1.5 Wildcards
1.6 Lo no permitido por Generics
1.7 Ejemplos

Contenido

1. Generics

1.1 Introducción
1.2 Tipos genéricos
1.3 Tipos raw
1.4 Métodos genéricos
1.5 Wildcards
1.6 Lo no permitido por Generics
1.7 Ejemplos
Ejemplo: Pila Generica
Ejemplo: Pila con Caja Generica
Ejemplo: Pila con Caja Generica y Wildcard

Introducción

¿Por qué Generics?

Introducción

¿Por qué Generics?

Generics

- Generics permite que, al definir una clase, interfaz o método, se puedan utilizar tipos (clases o interfaces) como parámetros
- Busca independizar el proceso del tipo de datos sobre los que se aplica
- Definición de modelos generales

Introducción

¿Por qué Generics?

Generics

- Generics permite que, al definir una clase, interfaz o método, se puedan utilizar tipos (clases o interfaces) como parámetros
- Busca independizar el proceso del tipo de datos sobre los que se aplica
- Definición de modelos generales

¿Y esto que acarrea?

Introducción

¿Por qué Generics?

Generics

- Generics permite que, al definir una clase, interfaz o método, se puedan utilizar tipos (clases o interfaces) como parámetros
- Busca independizar el proceso del tipo de datos sobre los que se aplica
- Definición de modelos generales

¿Y esto que acarrea?
Al utilizar parámetros en la definición de clases, interfaces y métodos, se provee una nueva manera de reutilizar código.

Introducción

Entonces, ¿cuáles son los beneficios de un código que usa Generics y otro que no?

Introducción

Entonces, ¿cuáles son los beneficios de un código que usa Generics y otro que no?

Beneficios

- Chequeos de tipos más rigurosos en tiempo de compilación
- Eliminación de casts.
- Implementación de algoritmos genéricos.

Introducción

Ejemplo 1, parte 1:

Listing 1: Clase genérica Caja

```
public class Caja<T> {
    private T contenido;
    public void setContenido(T contenido) {
        this.contenido = contenido;
    }
    public T getContenido() {
        return contenido;
    }
}
```

Introducción

Ejemplo 1, parte 2:

Listing 2: Uso de clase genérica Caja en main

```
public class Main {
    public static void main(String[] args) {
        Caja<String> cajaDeTextos = new Caja<>();
        cajaDeTextos.setContenido("Usando Generics");
        System.out.println(cajaDeTextos.getContenido());

        Caja<Integer> cajaDeNumeros = new Caja<>();
        cajaDeNumeros.setContenido(123);
        System.out.println(cajaDeNumeros.getContenido());
    }
}
```

Introducción

Adivine qué clase viene utilizando hace un tiempo y cumple con ser genérica...

Introducción

Listing 3: ArrayList es genérico

```java
public class Main {
    public static void main(String[] args) {
        ArrayList<String> nombres = new ArrayList<>();
        nombres.add("Ana");
        nombres.add("Juan");
        nombres.add("Luis");
        System.out.println(nombres);

        ArrayList<Integer> numeros = new ArrayList<>();
        numeros.add(10);
        numeros.add(20);
        numeros.add(30);
        System.out.println(numeros);
    }
}
```

Introducción

¿Se pueden utilizar tipos primitivos al utilizar Generics?

Introducción

¿Se pueden utilizar tipos primitivos al utilizar Generics?

Consideraciones

- No se pueden utilizar con primitivos pero sí con las clases que corresponden con ellos (clases wrapper).
- No se puede hacer un genérico tipo int pero si un Integer.
- Autoboxing a las clases wrappers (conversión automática que realiza el compilador entre un tipo de dato primitivo (como int) y su clase contenedora correspondiente (como Integer)).

Introducción

¿Qué relación puede existir entre sobrecarga de métodos y generics?

Introducción

¿Qué relación puede existir entre sobrecarga de métodos y generics?

Generics y Sobrecarga de métodos

- Si las operaciones realizadas por varios métodos sobrecargados son idénticas, pueden codificarse de manera compacta utilizando un método genérico.
- Con base en los tipos de los argumentos que se pasan al método genérico, el compilador maneja cada llamada al método de manera apropiada.
- Además de establecer las llamadas a los métodos, el compilador determina si las operaciones en el cuerpo del método se pueden aplicar a los elementos del tipo almacenado en el argumento de la clase genérica (arreglo, ArrayList).

Contenido

1. Generics

1.1 Introducción
1.2 Tipos genéricos
1.3 Tipos raw
1.4 Métodos genéricos
1.5 Wildcards
1.6 Lo no permitido por Generics
1.7 Ejemplos
Ejemplo: Pila Generica
Ejemplo: Pila con Caja Generica
Ejemplo: Pila con Caja Generica y Wildcard

Tipos genéricos

Ejemplo de clase No Genérica:

Listing 4: Clase NO genérica Caja

```
public class Caja {
    private Object objeto;
    public void setObjeto(Object objeto) {
        this.objeto = objeto;
    }
    public Object getObjeto() {
        return objeto;
    }
}
```

Tipos genéricos

¿Qué problemas se pueden presentar al utilizar la clase no genérica anterior?

Tipos genéricos

¿Qué problemas se pueden presentar al utilizar la clase no genérica anterior?

Problemas Clase NO Genérica

- No se puede verificar en tiempo de compilación como se utiliza la clase.
- Una parte del código puede ubicar un tipo de objeto en la Caja y esperar que se retorne un objeto de ese mismo tipo, mientras que, por error, en otra parte del código se le puede pasar otro tipo de objeto, generando un error en tiempo de ejecución.

Tipos genéricos

¿Cómo se declara una clase genérica?

Tipos genéricos

¿Cómo se declara una clase genérica?

Declaración Clase Genérica

- Al momento de declarar la clase, luego del nombre de ésta, se especifican los parámetros de tipos delimitados por < > .
- Los objetos a ser utilizados dentro de la clase se reemplazan por el tipo de parámetro de entrada.
- Estas variables de tipo pueden ser de cualquier tipo no primitivo.
- El mismo procedimiento puede utilizarse para declarar interfaces genéricas.

Tipos genéricos

¿Cuál sería la versión genérica del ejemplo previo?

Tipos genéricos

Sería precisamente el primer ejemplo visto

Tipos genéricos

Sería precisamente el primer ejemplo visto

Listing 6: Clase genérica Caja

```
public class Caja<T> {
    private T tipo;
    public void setTido(T tipo) {
        this.tipo = tipo;
    }
    public T getTipo() {
        return tipo;
    }
}
```

Tipos genéricos

Sería precisamente el primer ejemplo visto

Tipos genéricos

Sería precisamente el primer ejemplo visto

Listing 8: Clase genérica Caja

```
public class Caja<T> {
    private T tipo;
    public void setTido(T tipo) {
        this.tipo = tipo;
    }
    public T getTipo() {
        return tipo;
    }
}
```

Tipos genéricos

¿Cómo se declara una variable de tipo genérica?

Tipos genéricos

¿Cómo se declara una variable de tipo genérica?

Declaraciones variables genéricas

- `Caja<Integer> cajaEnteros;`
- `Caja<Double> cajaReales;`
- `Caja<String> cajaString;`
- `Caja< Caja<String> > cajaCajaString;`

Tipos genéricos

¿Cómo se declara una variable de tipo genérica?

Declaraciones variables genéricas

- `Caja<Integer> cajaEnteros;`
- `Caja<Double> cajaReales;`
- `Caja<String> cajaString;`
- `Caja< Caja<String> > cajaCajaString;`

¿Cómo se crea un objeto de tipo genérico?

Tipos genéricos

¿Cómo se declara una variable de tipo genérica?

Declaraciones variables genéricas

- `Caja<Integer> cajaEnteros;`
- `Caja<Double> cajaReales;`
- `Caja<String> cajaString;`
- `Caja< Caja<String> > cajaCajaString;`

¿Cómo se crea un objeto de tipo genérico?

Instanciar objetos tipo genéricos

- `Caja<Integer> cajaEnteros = new Caja<Integer>();`
- `Caja<Integer> cajaEnteros = new Caja<>();`

Tipos genéricos

¿Pero por qué es válido `Caja<Integer> cajaEnteros = new Caja<>();`?

Tipos genéricos

¿Pero por qué es válido `Caja<Integer> cajaEnteros = new Caja<>();`?

Instanciar objetos tipo genéricos

- Al invocar el constructor de una clase genérica se pueden reemplazar los argumentos de tipo con un conjunto vacío siempre y cuando el compilador pueda determinarlos o inferirlos a partir del contexto.
- Como este conjunto vacío se denota como < > se denomina notación diamante.

Tipos genéricos

¿Puede una clase tener múltiples parámetros de tipos?

Tipos genéricos

¿Puede una clase tener múltiples parámetros de tipos?

Múltiples parámetros de tipos

- Sí.
- `Terna<`T_1, T_2, T_3`> terna;`

Tipos genéricos

Ejercicio

Declare la clase genérica Terna (con 7 métodos, constructor, 3 getters y 3 setters) que implemente la interface ITerna. Posteriormente declare variables de tipo Terna e instáncielas utilizando distintos argumentos de tipo.

Tipos genéricos

Listing 9: Clase genérica Terna

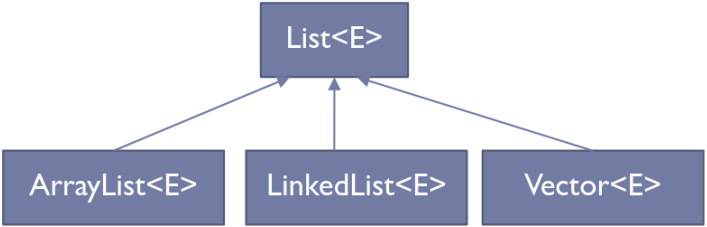
```
public interface ITerna<T1,T2,T3> {
   public T1 obtenerPrimero();
   public T2 obtenerSegundo();
   public T3 obtenerTercero();
}
public class Terna<T1,T2,T3> implements ITerna<T1,T2,T3> {
   private T1 primero;
   private T2 segundo;
   private T3 tercero;
   public Terna(T1 primero, T2 segundo, T3 tercero) {
      this.primero = primero;
      this.segundo = segundo;
      this.tercero = tercero ;
   }
   public T1 obtenerPrimero() {
      return primero;
   }
   public T2 obtenerSegundo() {
      return segundo;
   }
   public T3 obtenerTercero() {
      return tercero;
}
```

Tipos genéricos

¿Cuál sería una interfaz genérica estándar de Java?

Tipos genéricos

¿Cuál sería una interfaz genérica estándar de Java?



Tipos genéricos

Ejemplo

Listing 10: List genérico

```
public class TestArrayList {
    public static void main(String[] args) {
        List<Figura> lista = new ArrayList<Figura>();
        lista.add(new Circulo());
        Figura f = lista.get(0);
        Circulo c = (Circulo) lista.get(0);
    }
}
```

Tipos genéricos

¿Se pueden restringir los tipos genéricos?

Tipos genéricos

¿Se pueden restringir los tipos genéricos?

Restricción de tipos genéricos

- Se pueden restringir los tipos genéricos de tal manera que se pueda trabajar con un tipo específico y sus subtipos.
- Para establecer el límite superior, se coloca después del nombre del parámetro de tipo la palabra clave extends y el nombre de la clase o interfaz que representa dicha restricción.
- El extends se utiliza para clases e interfaces.

Tipos genéricos

Ejemplo

Listing 11: Restricción de tipo genérico en Java

```
public class CajaNumeros<T extends Number> {
    private T dato;
    public T darElemento() {
        return dato;
    }
    public void modificarDato(T dato) {
        this .dato = dato;
    }
}
```

Contenido

1. Generics

1.1 Introducción
1.2 Tipos genéricos
1.3 Tipos raw
1.4 Métodos genéricos
1.5 Wildcards
1.6 Lo no permitido por Generics
1.7 Ejemplos
Ejemplo: Pila Generica
Ejemplo: Pila con Caja Generica
Ejemplo: Pila con Caja Generica y Wildcard

Tipos raw

¿Qué es el tipo raw o crudo?

Tipos raw

¿Qué es el tipo raw o crudo?

Tipo raw

- Es el nombre de una clase o interfaz genérica cuando no se le pasa ningún argumento de tipo.
- `Caja cajaCruda = new Caja();`
- En este caso `Caja` es un tipo raw de `Caja<T>`.
- Un tipo de clase o interfaz no genérica no es un tipo raw.

Tipos raw

¿Qué advertencias se pueden presentar al utilizar tipos raw?

Tipos raw

¿Qué advertencias se pueden presentar al utilizar tipos raw?

Advertencias tipo raw

- Se presenta una advertencia ya que a un tipo parametrizado se le asigna un tipo raw.

Listing 14: Caja raw 1

```
Caja cajaCruda = new Caja();
Caja<Integer> intCaja = cajaCruda;
```

- Se presenta una advertencia pues el tipo raw se salta los chequeos de tipo genérico y así pasa su manejo de código inseguro a tiempo de ejecución.

Listing 15: Caja raw 2

```
Caja<String> stringCaja = new Caja<>();
Caja cajaCruda = stringCaja;
cajaCruda.modificar(8);
```

Contenido

1. Generics

1.1 Introducción
1.2 Tipos genéricos
1.3 Tipos raw
1.4 Métodos genéricos
1.5 Wildcards
1.6 Lo no permitido por Generics
1.7 Ejemplos
Ejemplo: Pila Generica
Ejemplo: Pila con Caja Generica
Ejemplo: Pila con Caja Generica y Wildcard

Métodos genéricos

¿Qué son los métodos genéricos?

Métodos genéricos

¿Qué son los métodos genéricos?

Métodos genéricos

- Son aquellos métodos que introducen su propio tipo de parámetros.
- Es similar a la declaración de tipo genérica, pero el alcance del parámetro está limitado al método donde se declara.
- Se admiten métodos genéricos estáticos y no estáticos, al igual que constructores genéricos.
- La sintaxis de un método genérico incluye un parámetro de tipo entre < > y aparece antes del tipo de retorno del método.

Métodos genéricos

```java
public class GenericMethodTest {
   // generic method printArray
   public static < E > void printArray( E[] inputArray ) {
      // Display array elements
      for(E element : inputArray) {
         System.out.print(element+" ");
      }
      System.out.println();
   }

   public static void main(String args[]) {
      // Create arrays of Integer, Double and Character
      Integer[] intArray = { 1, 2, 3, 4, 5 };
      Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4 };
      Character[] charArray = { 'H', 'E', 'L', 'L', 'O' };

      System.out.println("Array integerArray contains:");
      printArray(intArray);   // pass an Integer array

      System.out.println("\nArray doubleArray contains:");
      printArray(doubleArray);   // pass a Double array

      System.out.println("\nArray characterArray contains:");
      printArray(charArray);   // pass a Character array
   }
}
```

Métodos genéricos

¿Se puede utilizar el operador relacional (<, >, etc.) con tipos referenciados?

Métodos genéricos

¿Se puede utilizar el operador relacional (<, >, etc.) con tipos referenciados?

Operador relacional

- No, sin embargo, es posible comparar dos objetos de la misma clase, si esa clase implementa a la interfaz genérica Comparable<T>.
- Los objetos de clases que implementan Comparable<T> tienen el método compareTo(T t).
- Todas las clases de envoltura de tipos para tipos primitivos implementan a Comparable<T>.
- Un beneficio de implementar la interfaz genérica Comparable<T> es que los objetos de clases que implementan esta interfaz pueden utilizarse con métodos de ordenamiento y búsqueda sobre las clases Collections.

Métodos genéricos

Listing 16: Interfaz Comparable

```java
public class Libro implements Comparable<Libro> {
    private String titulo;
    private double precio;

    public Libro(String titulo, double precio) {
        this.titulo = titulo;
        this.precio = precio;
    }

    public String getTitulo() { return titulo; }
    public double getPrecio() { return precio; }

    @Override
    public int compareTo(Libro otroLibro) {
        return Double.compare(this.precio, otroLibro.precio);
    }
}
```

Métodos genéricos

Listing 17: Interfaz Comparable uso en Main

```java
public class Main {
    public static void main(String[] args) {
        List<Libro> libros = new ArrayList<>();
        libros.add(new Libro("Java Avanzado", 35.99));
        libros.add(new Libro("Python desde cero", 24.50));
        libros.add(new Libro("Introducción a la IA", 40.75));

        Collections.sort(libros);

        for (Libro libro : libros) {
            System.out.println(libro.getTitulo() + " - $" + libro.getPrecio());
        }
    }
}
```

Contenido

1. Generics

1.1 Introducción
1.2 Tipos genéricos
1.3 Tipos raw
1.4 Métodos genéricos
1.5 Wildcards
1.6 Lo no permitido por Generics
1.7 Ejemplos
Ejemplo: Pila Generica
Ejemplo: Pila con Caja Generica
Ejemplo: Pila con Caja Generica y Wildcard

Wildcards

¿Qué son los comodines o wildcards en Generics?

Wildcards

¿Qué son los comodines o wildcards en Generics?

Wildcards

- El signo de interrogación ?, también denominado comodín o wildcard representa un tipo no conocido.
- Un comodín de tipo parametrizado es una instancia de un tipo genérico donde al menos un argumento de tipo es un wildcard.
- Ejemplos: `Collection<?>`, `List<? extends Number>`, `Pair<String,?>`.
- Los comodines pueden utilizarse en gran variedad de situaciones, como tipo del parámetro, campo o variable local.
- Los wildcard nunca se utilizan como un argumento de tipo para una invocación de un método genérico o una creación de una instancia de una clase genérica.

Wildcards

Listing 18: Ejemplo Wildcard

```java
public class WildcardDemo {
    public static void imprimirLista(List<?> lista) {
        for (Object elem : lista) {
            System.out.println(elem);
        }
    }

    public static void main(String[] args) {
        List<String> palabras = Arrays.asList("hola", "mundo");
        List<Integer> numeros = Arrays.asList(1, 2, 3);

        imprimirLista(palabras);
        imprimirLista(numeros);
    }
}
```

Contenido

1. Generics

1.1 Introducción
1.2 Tipos genéricos
1.3 Tipos raw
1.4 Métodos genéricos
1.5 Wildcards
1.6 Lo no permitido por Generics
1.7 Ejemplos
Ejemplo: Pila Generica
Ejemplo: Pila con Caja Generica
Ejemplo: Pila con Caja Generica y Wildcard

Lo no permitido por Generics

¿Qué no permite Generics?

Lo no permitido por Generics

¿Qué no permite Generics?

Lo no permitido por Generics

- No se puede definir un miembro de una clase como estático genérico parametrizado. Cualquier intento de este tipo produce un error de compilación, pues hace una referencia estática a un tipo T no estático.
- No se pueden crear instancias de tipo T (tipo genérico).
- Generics no es compatible con tipos de datos primitivos. Aunque si se pueden utilizar las clases wrappers en lugar de los tipos de datos primitivos y luego usar los primitivos cuando se pasan los valores.
- No se puede crear una clase Excepción genérica.

Contenido

1. Generics

1.1 Introducción
1.2 Tipos genéricos
1.3 Tipos raw
1.4 Métodos genéricos
1.5 Wildcards
1.6 Lo no permitido por Generics
1.7 Ejemplos
Ejemplo: Pila Generica
Ejemplo: Pila con Caja Generica
Ejemplo: Pila con Caja Generica y Wildcard

Ejemplo: Pila Generica

- Para convertir nuestra clase `Pila` vista en la sesión, en una clase genérica que pueda manejar cualquier tipo de dato (no solo enteros), se debe usar genéricos (`<T>`) y cambiar el arreglo de `int[]` a un arreglo de `T[]`.

Ejemplo: Pila Generica

- Para convertir nuestra clase `Pila` vista en la sesión, en una clase genérica que pueda manejar cualquier tipo de dato (no solo enteros), se debe usar genéricos (`<T>`) y cambiar el arreglo de `int[]` a un arreglo de `T[]`.
- Como en Java no se pueden crear arreglos genéricos directamente, hay un truco con un arreglo de Object y un casting.

Ejemplo: Pila Generica

Listing 19: Ejemplo Pila Generica, parte 1

```
public class PilaGenerica<T> {
  private T[] arreglo;
  private int top;
  private int size;


  public PilaGenerica(int n) {
    this.arreglo = (T[]) new Object[n];
    this.top = 0;
    this.size = n;
}
```

Ejemplo: Pila Generica

Listing 20: Ejemplo Pila Generica, parte 2

```
public void Push(T x) throws IndexOutOfBoundsException {
   if(this.top < this.size) {
     this.arreglo[top] = x;
     this.top++;
   } else {
     throw new StackOverflowError();
   }
  }
```

Ejemplo: Pila Generica

Listing 21: Ejemplo Pila Generica, parte 3

```
public T Pop() throws IndexOutOfBoundsException {
  if(this.top > 0) {
    this.top--;
    return this.arreglo[top];
  } else {
    throw new IndexOutOfBoundsException();
  }
}
```

Ejemplo: Pila Generica

Listing 22: Ejemplo Pila Generica, parte 4

```
public T[] getArreglo() {
    return arreglo;
  }

  public int getTop() {
    return top;
  }

  public int getSize() {
    return size;
  }
```

Ejemplo: Pila Generica

Listing 23: Ejemplo Pila Generica, parte 5

```
public void setArreglo(T[] arreglo) {
  this.arreglo = arreglo;
}

public void setTop(int top) {
  this.top = top;
}

public void setSize(int size) {
  this.size = size;
}
```

Ejemplo: Pila con Caja Generica

Listing 24: Ejemplo Pila Generica Caja, parte 1

```
public class PilaCaja<T> {
  private Caja<T>[] arreglo;
  private int top;
  private int size;

  @SuppressWarnings("unchecked")
  public PilaCaja(int n) {
    this.arreglo = (Caja<T>[]) new Caja[n]; // arreglo genérico
    this.top = 0;
    this.size = n;
  }
```

Ejemplo: Pila con Caja Generica

Listing 25: Ejemplo Pila Generica Caja, parte 2

```
public void Push(Caja<T> caja) throws IndexOutOfBoundsException {
   if (this.top < this.size) {
     this.arreglo[top] = caja;
     this.top++;
   } else {
     throw new StackOverflowError();
   }
  }
```

Ejemplo: Pila con Caja Generica

Listing 26: Ejemplo Pila Generica Caja, parte 3

```
public Caja<T> Pop() throws IndexOutOfBoundsException {
   if (this.top > 0) {
     this.top--;
     return this.arreglo[top];
   } else {
     throw new IndexOutOfBoundsException();
   }
  }
```

Ejemplo: Pila con Caja Generica

Listing 27: Ejemplo Pila Generica Caja, parte 4

```
public Caja<T>[] getArreglo() {
    return arreglo;
  }

  public int getTop() {
    return top;
  }

  public int getSize() {
    return size;
  }
```

Ejemplo: Pila con Caja Generica

Listing 28: Ejemplo Pila Generica Caja, parte 5

```
public void setArreglo(Caja<T>[] arreglo) {
  this.arreglo = arreglo;
}

public void setTop(int top) {
  this.top = top;
}

public void setSize(int size) {
  this.size = size;
}
```

Ejemplo: Pila con Caja Generica y Wildcard

Para crear una clase `PilaCajaGenerica` que sea una pila que no sea genérica en sí misma, pero cuyo contenido sea objetos de la clase genérica `Caja<T>`, lo que se puede hacer es:

Ejemplo: Pila con Caja Generica y Wildcard

Para crear una clase `PilaCajaGenerica` que sea una pila que no sea genérica en sí misma, pero cuyo contenido sea objetos de la clase genérica `Caja<T>`, lo que se puede hacer es:

- Declarar la pila con arreglo de `Caja<?> []` para poder contener cajas con cualquier tipo (? es wildcard).

Ejemplo: Pila con Caja Generica y Wildcard

Para crear una clase `PilaCajaGenerica` que sea una pila que no sea genérica en sí misma, pero cuyo contenido sea objetos de la clase genérica `Caja<T>`, lo que se puede hacer es:

- Declarar la pila con arreglo de `Caja<?> []` para poder contener cajas con cualquier tipo (? es wildcard).
- Así la pila almacena distintas `Caja<T>`, cada una con diferente tipo, como `Caja<String>`, `Caja<Integer>`, etc.

Ejemplo: Pila con Caja Generica y Wildcard

Para crear una clase `PilaCajaGenerica` que sea una pila que no sea genérica en sí misma, pero cuyo contenido sea objetos de la clase genérica `Caja<T>`, lo que se puede hacer es:

- Declarar la pila con arreglo de `Caja<?> []` para poder contener cajas con cualquier tipo (? es wildcard).
- Así la pila almacena distintas `Caja<T>`, cada una con diferente tipo, como `Caja<String>`, `Caja<Integer>`, etc.
- Los métodos `Push` y `Pop` manipulan objetos `Caja<?>`.

Ejemplo: Pila con Caja Generica y Wildcard

Listing 29: Ejemplo Pila con Caja Generica y Wildcard, parte 1

```java
public class PilaCajaGenericaWildcard {
  private Caja<?>[] arreglo;
  private int top;
  private int size;

  public PilaCajaGenericaWildcard(int n) {
    this.arreglo = (Caja<?>[]) new Caja<?>[n];
    this.top = 0;
    this.size = n;
  }
}
```

Ejemplo: Pila con Caja Generica y Wildcard

Listing 30: Ejemplo Pila con Caja Generica y Wildcard, parte 2

```
public void Push(Caja<?> caja) throws IndexOutOfBoundsException {
   if (this.top < this.size) {
     this.arreglo[top] = caja;
     this.top++;
   } else {
     throw new StackOverflowError();
   }
  }
```

Ejemplo: Pila con Caja Generica y Wildcard

Listing 31: Ejemplo Pila con Caja Generica y Wildcard, parte 3

```
public Caja<?> Pop() throws IndexOutOfBoundsException {
   if (this.top > 0) {
     this.top--;
     return this.arreglo[top];
   } else {
     throw new IndexOutOfBoundsException();
   }
  }
```

Ejemplo: Pila con Caja Generica y Wildcard

Listing 32: Ejemplo Pila con Caja Generica y Wildcard, parte 4

```
public Caja<?>[] getArreglo() {
    return arreglo;
  }

  public int getTop() {
    return top;
  }

  public int getSize() {
    return size;
  }
```

Ejemplo: Pila con Caja Generica y Wildcard

Listing 33: Ejemplo Pila con Caja Generica y Wildcard, parte 5

```
public void setArreglo(Caja<?>[] arreglo) {
  this.arreglo = arreglo;
}

public void setTop(int top) {
  this.top = top;
}

public void setSize(int size) {
  this.size = size;
}
```